

Globally-Mapped LiDAR Navigation for a Quadruped Robot

Implementation and Extension of a Classical Mapping–Localization–Planning Stack

Sanjit K.*

Southern Methodist University

akpostme@gmail.com

June 22, 2026

Abstract

This report documents the implementation and extension of a classical navigation stack for a quadruped robot operating on uneven terrain, following the LiDAR mapping $\rightarrow$ localization $\rightarrow$ global planning $\rightarrow$ local obstacle avoidance architecture of a recent IEEE study [1]. We build the stack from first principles in Python (`numpy/matplotlib` only [2, 3]): a 2.5D elevation-grid discretization of raw point clouds; a per-cell Kalman-filtered probabilistic terrain map [4, 5]; a Dijkstra/A* global planner [6, 7] with elastic-band (Timed-Elastic-Band) smoothing [8]; and an artificial-potential-field (APF) local planner in the Khatib formulation [9]. We validate on synthetic terrain, on real automotive LiDAR from the KITTI dataset [10], and on an indoor RGB-D reconstruction [11], and we provide a ROS 2 / Nav2 integration path [12, 13]. Beyond reproduction, we add a moving-goal APF mode aimed at the autonomous *person-following* task [14]. Experiments expose three practical lessons: (i) traversability thresholds are sensor-specific, with clean LiDAR working at default settings while noisy single-view RGB-D requires looser tolerances and stronger smoothing; (ii) naive multi-scan fusion without scan registration (ICP [15]) inflates obstacles through ghosting, demonstrating empirically why the reference method requires a localization stage; and (iii) APF alone is trapped by local minima at large obstacles, confirming the necessity of the two-level (global + local) design.

1 Introduction

Legged robots are increasingly deployed on terrain that wheeled platforms cannot negotiate—construction sites, disaster zones, and natural ground with steps, slopes, and gaps. Autonomous navigation for such robots requires a terrain representation that preserves the geometric features relevant to footing, a means of localizing within that representation, and planners that produce both a long-range route and reactive avoidance of nearby hazards.

The reference study [1] proposes an integrated stack for a quadruped that combines established components: a LiDAR-built terrain elevation map refined by Kalman filtering; SLAM-based localization fusing GPS, IMU, odometry, and LiDAR with Iterative Closest Point (ICP) scan matching [15]; a Dijkstra global planner [6] on a grid map smoothed by a Timed-Elastic-Band (TEB) optimizer [8]; and an artificial-potential-field (APF) local obstacle avoider [9]. The contribution of [1] is the *integration* rather than any single new algorithm.

*Undergraduate research intern. Edit author/advisor fields before submission.

This report reproduces the core of that stack and extends it toward the research direction of interest—autonomous *person-following* [14]—where the navigation target is a moving person rather than a fixed goal. Our contributions are:

1. A self-contained 2.5D elevation-grid discretization of raw point clouds with robust per-cell statistics and a traversability cost model (Sec. 4).
2. A per-cell Kalman probabilistic terrain map that fuses multiple scans and gates untrustworthy cells by variance (Sec. 5).
3. Dijkstra/A* global planning, elastic-band smoothing, and a Khatib APF local planner with a moving-goal (person-following) mode (Secs. 6–7).
4. A ROS 2/Nav2 deployment path that publishes the map as a costmap layer (Sec. 8).
5. An empirical study on synthetic, automotive-LiDAR, and indoor-RGB-D data that surfaces concrete lessons about sensor-specific tuning, scan registration, and global/local planner roles (Sec. 9).

2 Background and Related Work

Terrain representations. Occupancy grids [16] discretize the environment into free/occupied cells but collapse the vertical dimension. For legged robots the *2.5D elevation grid*—one height value (and optionally a variance) per planar cell—retains steps, curbs, and slopes while remaining far cheaper than full 3D volumetric maps such as OctoMap [17]. Robot-centric, uncertainty-aware elevation mapping was formalized by Fankhauser *et al.* [18, 5], who maintain a Gaussian height estimate per cell updated by a Kalman filter [4, 19].

Planning. Dijkstra’s algorithm [6] computes shortest-cost paths by uniform-cost expansion; A* [7] adds an admissible heuristic for efficiency. Grid paths are jagged, so trajectory optimizers such as TEB [20, 8] refine them under geometric and (for full TEB) kinodynamic constraints. For reactive avoidance, the artificial potential field [9] treats the goal as an attractor and obstacles as repellers; its well-known weakness is entrapment in local minima.

Datasets and tooling. We evaluate on the KITTI autonomous-driving benchmark [21, 10] (Velo-dyne HDL-64 scans) and on an indoor RGB-D fragment from Open3D [11]. Deployment targets ROS 2 and the Nav2 navigation framework [22, 12, 13].

3 System Overview

Figure 1 shows the pipeline. Raw point clouds, assumed expressed in a stable world frame by the localization stage, are fused into a probabilistic 2.5D elevation map. The map yields a traversability cost grid consumed by the global planner; the resulting route is smoothed and tracked by the APF local planner, which also supports a moving goal for person-following. The localization block (SLAM/ICP) is treated as an assumed input in this work (Sec. 11).

4 2.5D Elevation Discretization

A point cloud $\{(x_k, y_k, z_k)\}_{k=1}^N$ is binned into a grid of resolution r . For a cell with the set of contained returns $\mathcal{Z}$, we accumulate count, sum, and sum-of-squares in a single vectorized pass and

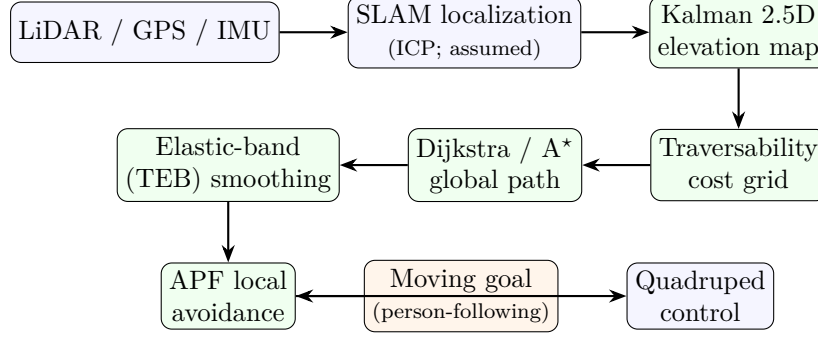


Figure 1: System architecture. Green blocks are implemented in this work; the orange block is the person-following extension; the localization block (white) is assumed. The structure follows the reference stack [1].

Table 1: Default quadruped mobility parameters (Go2/Spot class).

Symbol	Meaning	Default
ρ_{body}	footprint radius (inflation)	0.30 m
Δ_{max}	maximum climbable step	0.18 m
θ_{max}	maximum traversable slope	30°
σ_{max}	maximum within-cell height std	0.05 m
r	grid resolution	0.10 m

derive a *robust* per-cell elevation and roughness:

$$\hat{h} = \frac{1}{|\mathcal{Z}|} \sum_{z \in \mathcal{Z}} z, \quad \sigma_{\text{cell}} = \sqrt{\mathbb{E}[z^2] - \mathbb{E}[z]^2}. \quad (1)$$

Using the standard deviation (1) rather than the raw range $\max z - \min z$ makes roughness insensitive to a single stray return and to point density. Slope is the magnitude of the elevation gradient, computed in a NaN-aware manner so cells adjacent to map holes are not corrupted:

$$s = \|\nabla \hat{h}\|, \quad \Delta = \max_{j \in \mathcal{N}_4} |\hat{h} - \hat{h}_j|, \quad (2)$$

where Δ is the maximum height difference to the four-connected neighbours (a discrete step detector, following the grid height-difference test of [1]). A cell is classified **LETHAL** if it exceeds the robot’s mobility limits (Table 1):

$$\text{LETHAL} \iff \Delta > \Delta_{\text{max}} \vee s > \tan \theta_{\text{max}} \vee \sigma_{\text{cell}} > \sigma_{\text{max}}. \quad (3)$$

Isolated lethal cells (sensor speckle) are removed by dropping connected components smaller than three cells—this, unlike morphological opening, does not thin genuine thin walls. Surviving obstacles are dilated by the footprint radius for clearance. The continuous cost combines smooth penalties with a lethal sentinel:

$$c = 1 + w_s \frac{s}{\tan \theta_{\text{max}}} + w_r \frac{\sigma_{\text{cell}}}{\sigma_{\text{max}}}, \quad c \leftarrow \infty \text{ if LETHAL}. \quad (4)$$

5 Probabilistic Terrain Mapping with a Kalman Filter

Each cell maintains a height estimate $\hat{h}$ and variance P . A LiDAR return at range ϱ from the sensor is assigned a measurement variance that grows with range,

$$R = \sigma_0^2 + \alpha \varrho^2, \quad (5)$$

reflecting the degradation of LiDAR depth accuracy with distance. Multiple returns landing in one cell during a single scan are combined in information form before the update,

$$\bar{z} = \frac{\sum_k z_k / R_k}{\sum_k 1 / R_k}, \quad \bar{R} = \frac{1}{\sum_k 1 / R_k}. \quad (6)$$

The cell is then updated by the scalar Kalman filter [4, 19, 5]:

$$(\text{init}, P = \infty): \quad \hat{h} \leftarrow \bar{z}, \quad P \leftarrow \bar{R}, \quad (7)$$

$$(\text{update}): \quad K = \frac{P}{P + \bar{R}}, \quad \hat{h} \leftarrow \hat{h} + K(\bar{z} - \hat{h}), \quad P \leftarrow (1 - K)P. \quad (8)$$

A prediction step $P \leftarrow P + q$ injects process noise so the map remains adaptive to change. Listing 1 shows the implementation core. A subtle but important detail: the prior-finite mask must be captured *before* mutating the height array, otherwise a cell’s first observation is both initialized and fused against itself, incorrectly halving its variance.

Listing 1: Per-cell Kalman fusion (information-form combine then scalar update).

```
had = np.isfinite(self.height) # snapshot BEFORE mutating
fresh = observed & ~had # first observation -> initialize
self.height[fresh] = z_meas[fresh]
self.variance[fresh] = R_meas[fresh]
upd = observed & had # fuse with existing estimate
P, R = self.variance[upd], R_meas[upd]
K = P / (P + R)
self.height[upd] += K * (z_meas[upd] - self.height[upd])
self.variance[upd] = (1.0 - K) * P
```

The fused map is converted to a cost grid via Eqs. (2)–(4), with the addition of a *variance gate*: cells whose variance remains above a threshold are labelled UNKNOWN rather than trusted, so the planner does not commit to poorly observed ground.

6 Global Path Planning and Smoothing

Following [1] we plan globally with Dijkstra’s algorithm [6] over the cost grid; we also provide A* [7] with the octile heuristic for efficiency. Both are 8-connected, and the cost of traversing from cell u to neighbour v is the mean cell cost scaled by the step distance:

$$g(u \rightarrow v) = d(u, v) \cdot \frac{1}{2}(c_u + c_v), \quad d \in \{1, \sqrt{2}\}. \quad (9)$$

Dijkstra is exactly A* with a zero heuristic; a unit test confirms the two return paths of identical cost. The raw grid path is then geometrically smoothed by an elastic band—the geometric core of TEB [8]. Each interior waypoint p_i is pulled toward the midpoint of its neighbours (contraction)

and pushed away from obstacles within an influence distance via the gradient of a chamfer distance field D :

$$p_i \leftarrow p_i + \underbrace{w_c(p_{i-1} + p_{i+1} - 2p_i)}_{\text{smoothness}} + \underbrace{w_o \max\left(0, \frac{d_0 - D(p_i)}{d_0}\right)}_{\text{clearance}} \widehat{\nabla D}. \quad (10)$$

Endpoints are fixed and any waypoint that would enter an obstacle cell is rejected. We note explicitly that this is the geometric stand-in for TEB; the full time-optimal, kinodynamic TEB of [8] is left to future work.

7 Local Obstacle Avoidance (APF) and Person-Following

The local planner uses the Khatib potential field [9]. With robot position q and goal q_g , the attractive force (with a conic far-field cap to avoid huge forces at long range) and the repulsive force from the nearest obstacle at distance ϱ within influence radius ϱ_0 are

$$F_{\text{att}} = k_a (q_g - q), \quad (11)$$

$$F_{\text{rep}} = \begin{cases} k_r \left(\frac{1}{\varrho} - \frac{1}{\varrho_0} \right) \frac{1}{\varrho^2} \widehat{\nabla \varrho}, & \varrho \leq \varrho_0, \\ 0, & \varrho > \varrho_0, \end{cases} \quad (12)$$

and the robot integrates $q \leftarrow q + \text{clip}(F_{\text{att}} + F_{\text{rep}}, v_{\text{max}}) \Delta t$. Obstacle distance ϱ and its gradient come from a chamfer distance field, so each force query is $O(1)$.

Two-level operation. Pure APF toward a distant goal is trapped by local minima at large obstacles (Sec. 9). The faithful architecture therefore tracks the global path with a moving *carrot* sub-goal a fixed look-ahead ahead of the robot, so the attractive force never points through a wall while repulsion still trims hazards.

Person-following. For the target application the goal is a moving person, modelled as $q_g(t)$, a callable of time [14]. The same APF loop tracks $q_g(t)$ directly while repulsion avoids terrain hazards; the global Dijkstra planner becomes secondary because the destination is no longer fixed.

8 ROS 2 / Nav2 Integration

For deployment, a ROS 2 (Humble) node subscribes to a `PointCloud2`, discretizes it, and republishes a `nav_msgs/OccupancyGrid` that Nav2 [12, 13] consumes through a `StaticLayer`. To avoid double inflation, the node publishes the un-inflated traversability grid and lets Nav2’s `InflationLayer` apply footprint clearance; the soft slope/roughness costs are preserved by setting `trinary_costmap: false`. This keeps the numpy core unchanged and reuses Nav2’s mature planners and controllers.

9 Experiments and Results

All components are implemented in Python with only `numpy` [2] and `matplotlib` [3] (no `scipy`/ROS dependency for the core), and validated by 21 unit tests.

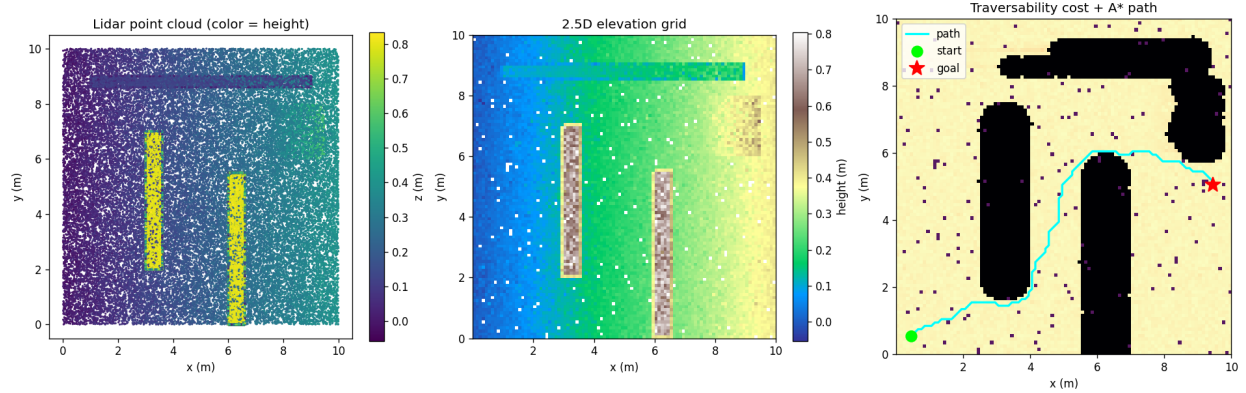


Figure 2: Synthetic scene. Left: raw point cloud coloured by height. Centre: 2.5D elevation grid. Right: traversability cost grid with the A^{*} path routing between the two walls.

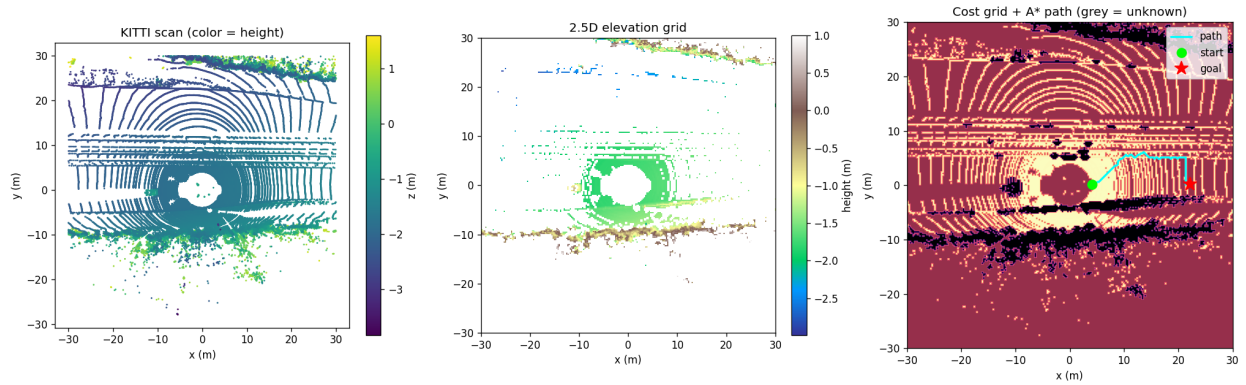


Figure 3: Real KITTI scan (000001.bin). Left: scan coloured by height (Velodyne rings visible). Centre: elevation grid. Right: cost grid with the planned path; grey is unobserved space.

9.1 Synthetic terrain

Figure 2 shows the discretization pipeline on a synthetic 10 × 10 m scene with a sloped ground plane, two walls, a climbable curb, and a ramp. The planner routes between the walls; the elevation and cost layers correctly separate traversable ground from lethal obstacles (Table 2, row 1).

9.2 Real automotive LiDAR (KITTI)

On a Velodyne HDL-64 scan from KITTI [10] (Fig. 3, ~120k points), the pipeline produces a sensible road path *using default parameters*: the sensor sits ~1.7 m above the ground, coverage is sparse at range (most far cells are UNKNOWN), and obstacles such as vehicles and walls are correctly marked lethal (Table 2). Clean LiDAR requires no tuning.

9.3 Real indoor RGB-D and sensor-specific tuning

On an Open3D indoor room fragment [11] (Fig. 4), the vertical axis is not gravity-aligned; we auto-detect it from the dominant height-histogram peak (the floor) and reorient. Critically, a single-view RGB-D floor exhibits ~0.15 m to 0.2 m of depth scatter *per cell* at grazing angles, so the default

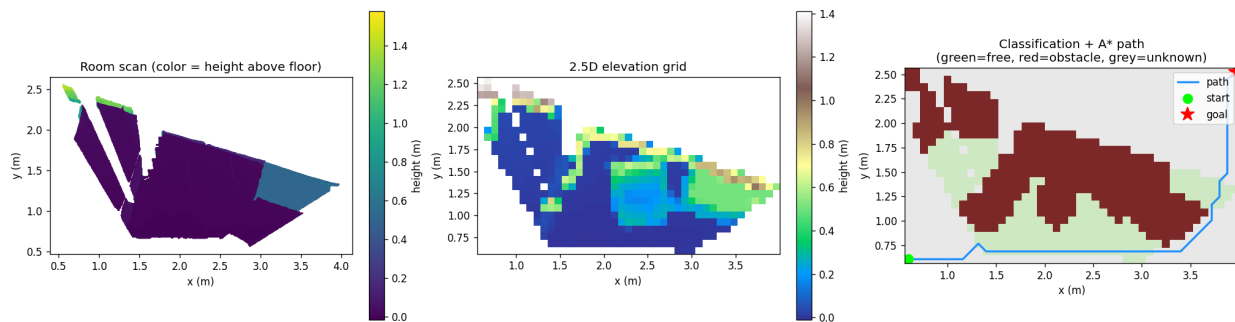


Figure 4: Indoor RGB-D room scan. Left: scan coloured by height above floor. Centre: elevation grid. Right: classification (green free, red obstacle, grey unknown) with the path routing around furniture.

Table 2: Per-scan discretization and planning results.

Scene	Points	Obs. cells	Lethal	Unknown	Path
Synthetic	85 002	9 837	2 749	163	13.3 m
KITTI 000001	120 268	14 027	5 233	43 573	27.6 m
KITTI 000002	126 891	4 429	2 263	53 171	26.3 m
RGB-D room	196 133	528	325	547	4.97 m

LiDAR thresholds (correctly but unhelpfully) flag the floor as low-confidence. Loosening $\sigma_{\max}$ and $\theta_{\max}$ and increasing surface smoothing recovers a traversable floor (1.30 m², Table 2) while keeping $\sim 80\%$ of furniture cells lethal. **Lesson:** traversability thresholds encode sensor *and* robot tolerance and must be matched to the sensor’s noise.

9.4 Kalman fusion and the need for registration

Figure 5 (left) shows the mean per-cell variance falling monotonically as scans are fused—the noise reduction the reference method attributes to Kalman filtering [1, 5]. To test *multi-scan* fusion on real data we fused the two provided KITTI scans. Table 3 reports a control: fusing a scan with itself (perfectly aligned) leaves the lethal-cell count unchanged and merely lowers variance, whereas fusing two consecutive scans inflates lethal cells by 80 %. The cause is a pose difference (near-sensor ground at -1.67 m vs. -0.88 m): without scan registration via ICP [15], fusing differently-posed scans produces ghosting. **This is an empirical demonstration of why the reference stack includes a localization stage**—our map assumes known poses, which the SLAM/ICP block must supply.

9.5 Full stack and person-following

Figure 5 shows the complete fixed-goal stack: Kalman map, Dijkstra global path (orange), elastic-band smoothing (navy), and APF tracking (blue). Pure APF toward the far goal stalled 4.4 m short in a local minimum at the walls; the carrot-following two-level scheme reaches the goal within the 0.15 m tolerance while maintaining 0.57 m obstacle clearance. Figure 6 shows the person-following

Table 3: Kalman fusion control on real KITTI scans (0.25 m grid).

Fusion	Lethal cells	Observed cells
Scan 1 alone	4 108	14 027
Scan 1 + Scan 1 (aligned)	4 108	14 027
Scan 1 + Scan 2 (diff. pose)	7 380	15 910

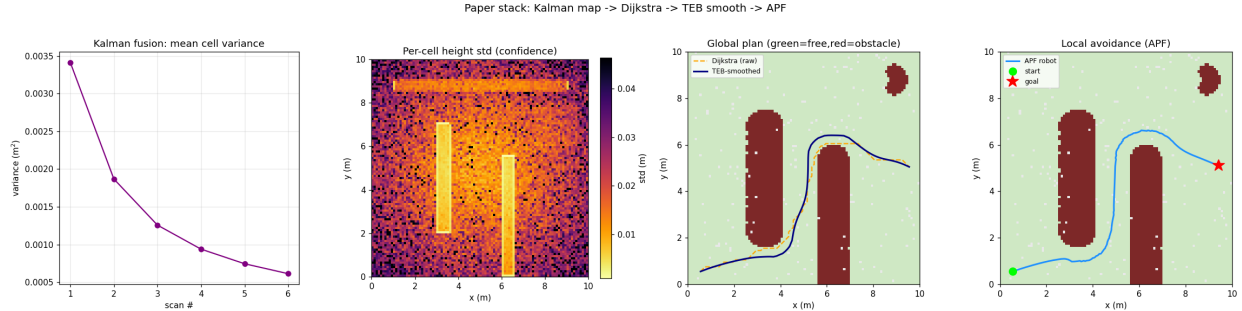


Figure 5: Paper stack on synthetic terrain. Left: Kalman variance decreasing over fused scans. Centre-left: per-cell height standard deviation (confidence). Centre-right: Dijkstra path (orange) and elastic-band-smoothed path (navy). Right: APF trajectory tracking the smoothed global path.

mode: a person walks a guaranteed-free route and the robot APF-tracks them at a mean gap of 0.58 m while avoiding hazards [14].

9.6 Three-dimensional visualization

To aid interpretation, the elevation grid is rendered as a 3D surface coloured by traversability with the path draped on top (Fig. 7); an interactive WebGL version is also produced. The relief makes the relationship between the route and the standing obstacles immediately legible.

10 Discussion

The experiments yield three transferable findings. First, the same code base handles clean automotive LiDAR at default settings but requires looser tolerances and stronger smoothing for noisy single-view RGB-D; traversability thresholds are not universal constants but sensor-conditioned. Second, the fusion control (Table 3) makes concrete why globally-consistent localization is a prerequisite for multi-scan mapping: without ICP registration [15], fusion degrades rather than improves the map. Third, the APF local-minimum result validates the two-level design of the reference stack [1]—a global planner is not optional decoration but the mechanism that frees the local planner from terrain traps.

11 Limitations and Future Work

The principal omission is the SLAM/ICP *localization* block: poses are assumed known and passed to the map. Closing this loop with a point-to-plane ICP [15] or an off-the-shelf LiDAR-inertial odometry front-end would enable true multi-scan fusion on moving-robot data. The smoother is the geometric core of TEB, not the full time-optimal optimizer [8]. For person-following, a perception

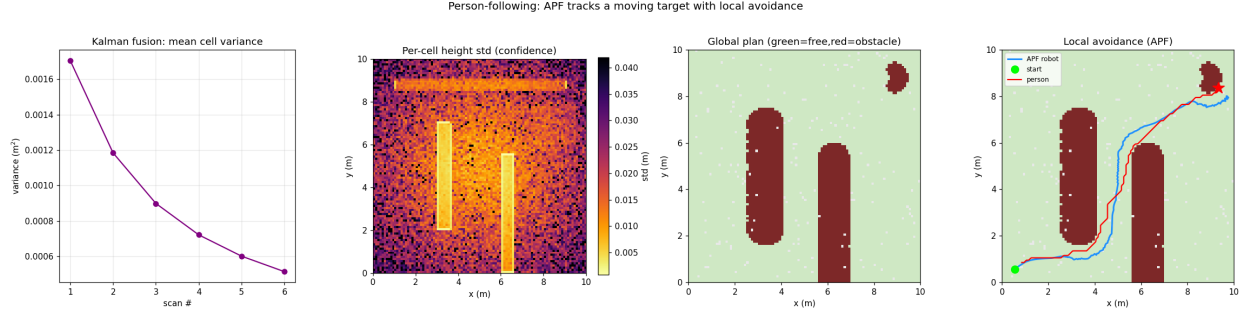


Figure 6: Person-following. The robot (blue) tracks a moving person (red) with APF local avoidance around the two walls, at a mean separation of 0.58 m.

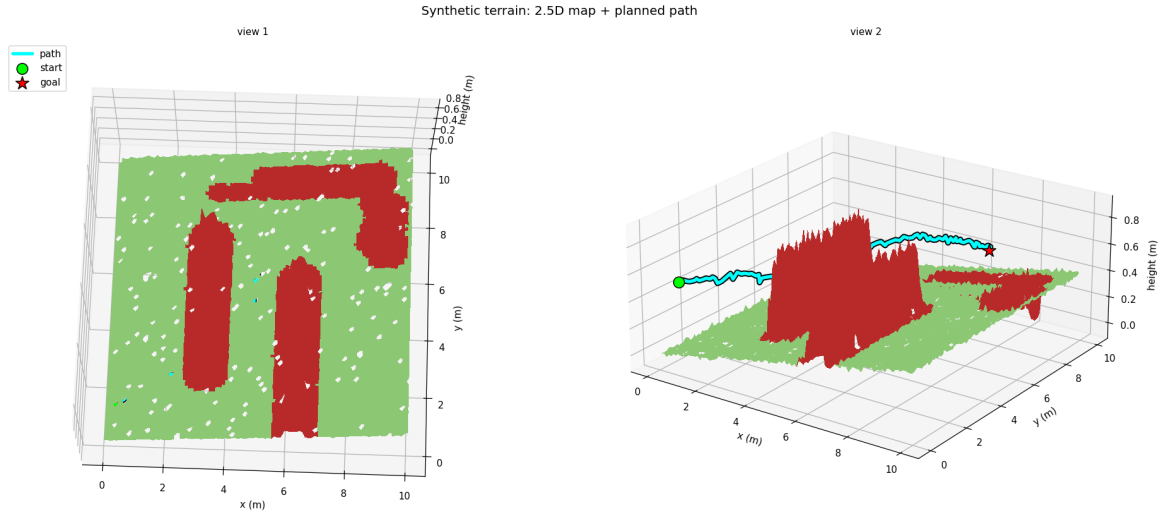


Figure 7: 3D view of the synthetic 2.5D map (two angles). Green is traversable, red is obstacles (height exaggerated 4×); the cyan tube is the planned path from the green start to the red goal.

front-end (RGB person detection, tracking, and target-position estimation [14]) is required to supply $q_g(t)$. Finally, on-robot validation through the Nav2 integration [12] remains to be done.

12 Conclusion

We reproduced and extended a classical mapping–localization–planning stack for quadruped navigation [1], implementing a Kalman-filtered 2.5D terrain map, Dijkstra/A* global planning, elastic-band smoothing, and a Khatib APF local planner with a person-following mode, plus a Nav2 deployment path. Validation on synthetic, automotive-LiDAR, and indoor-RGB-D data confirmed the stack functions on real input and surfaced practical lessons on sensor-specific tuning, the necessity of scan registration, and the global/local planner division of labour.

Reproducibility

The implementation is plain Python (`numpy/matplotlib`). Figures in this report are produced directly by the project’s demo and example scripts; all quantitative results are emitted by those scripts and covered by the unit-test suite.

References

- [1] Y. Liu, L. Jiang, F. Zou, B. Xing, Z. Wang, and B. Su, “Research on path planning of quadruped robot based on globally mapping localization,” in *2020 IEEE International Conference on Unmanned Systems (ICUS)*, pp. 346–351, 2020.
- [2] C. R. Harris, K. J. Millman, S. J. van der Walt, *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [3] J. D. Hunter, “Matplotlib: A 2d graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [4] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [5] P. Fankhauser, M. Bloesch, and M. Hutter, “Probabilistic terrain mapping for mobile robots with uncertain localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 3019–3026, 2018.
- [6] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [7] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [8] C. Rösmann, F. Hoffmann, and T. Bertram, “Integrated online trajectory planning and optimization in distinctive topologies,” *Robotics and Autonomous Systems*, vol. 88, pp. 142–153, 2017.
- [9] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [10] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, “Vision meets robotics: The KITTI dataset,” *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [11] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A modern library for 3D data processing,” *arXiv preprint arXiv:1801.09847*, 2018.
- [12] S. Macenski, F. Martín, R. White, and J. G. Clavero, “The marathon 2: A navigation system,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2718–2725, 2020.
- [13] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

- [14] M. J. Islam, J. Hong, and J. Sattar, “Person-following by autonomous robots: A categorical overview,” *The International Journal of Robotics Research*, vol. 38, no. 14, pp. 1581–1618, 2019.
- [15] P. J. Besl and N. D. McKay, “A method for registration of 3-d shapes,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992.
- [16] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” *Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [17] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, “OctoMap: An efficient probabilistic 3D mapping framework based on octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [18] P. Fankhauser, M. Bloesch, C. Gehring, M. Hutter, and R. Siegwart, “Robot-centric elevation mapping with uncertainty estimates,” in *Mobile Service Robotics (CLAWAR)*, pp. 433–440, 2014.
- [19] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [20] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, “Trajectory modification considering dynamic constraints of autonomous robots,” in *7th German Conference on Robotics (ROBOTIK)*, pp. 74–79, 2012.
- [21] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? the KITTI vision benchmark suite,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3354–3361, 2012.
- [22] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “ROS: An open-source robot operating system,” in *ICRA Workshop on Open Source Software*, 2009.